

Structured Bindings can introduce a Pack

Document #: P1061R8
Date: 2024-04-14
Project: Programming Language C++
Audience: CWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>
Jonathan Wakely
<cxx@kayari.org>

Contents

- [1 Revision History](#)
- [2 Motivation](#)
- [3 Proposal](#)
 - [3.1 Other Languages](#)
 - [3.2 Implementation Burden](#)
 - [3.3 Implementation Experience](#)
 - [3.4 Handling non-dependent packs in the wording](#)
 - [3.4.1 The Issaquah Example](#)
 - [3.4.2 The Varna Example](#)
 - [3.4.3 More Templates](#)
 - [3.5 Namespace-scope packs](#)
- [4 Wording](#)
 - [4.1 Feature-Test Macro](#)
- [5 Acknowledgements](#)
- [6 References](#)

§ 1 Revision History

R8 re-adds the [namespace-scope exclusion](#), and more wording updates. Also rebases the wording to account for [\[P0609R3\]](#).

R7 attempts to word the post-Varna version.

R6 has added wording changes and adds some more complicated examples to motivate how to actually word this paper.

R5 has minor wording changes.

R4 significantly improves the wording after review in Issaquah.

R3 removes the exclusion of namespace-scope per [EWG guidance](#).

R2 adds a section about implementation complexity, implementation experience, and wording.

R1 of this paper [P1061R1] was presented to EWG in Belfast 2019 [P1061R1.Minutes] which approved the direction as presented (12-5-2-0-1).

R0 of this paper [P1061R0] was presented to EWGI in Kona 2019 [P1061R0.Minutes], who reviewed it favorably and thought this was a good investment of our time (4-3-4-1-0). The consensus in the room was that the restriction that the introduced pack need not be the trailing identifier.

§ 2 Motivation

Function parameter packs and tuples are conceptually very similar. Both are heterogeneous sequences of objects. Some problems are easier to solve with a parameter pack, some are easier to solve with a tuple. Today, it's trivial to convert a pack to a tuple, but it's somewhat more involved to convert a tuple to a pack. You have to go through `std::apply()` [N3915]:

```
std::tuple<A, B, C> tup = ...;
std::apply([&](auto&&... elems){
    // now I have a pack
}, tup);
```

This is great for cases where we just need to call a [non-overloaded] function or function object, but rapidly becomes much more awkward as we dial up the complexity. Not to mention if I want to return from the outer scope based on what these elements have to be.

How do we compute the dot product of two tuples? It's a choose your own adventure of awkward choices:

Nested <code>apply()</code>	Using <code>index_sequence</code>
<pre>template <class P, class Q> auto dot_product(P p, Q q) { return std::apply([&](auto... p_elems) { return std::apply([&](auto... q_elems) { return (... + (p_elems * q_elems)); }, q); }, p); }</pre>	<pre>template <size_t... Is, class P, class Q> auto dot_product(std::index_sequence<Is...>, P p, Q, q) { return (... + (std::get<Is>(p) * std::get<Is>(q))); } template <class P, class Q> auto dot_product(P p, Q q) { return dot_product(std::make_index_sequence<std::tuple_size<P>::value> {}), p, q); }</pre>

Regardless of which option you dislike the least, both are limited to only `std::tuples`. We don't have the ability to do this at all for any of the other kinds of types that can be used in a structured binding declaration [P0144R2] - because we need to explicit list the correct number of identifiers, and we might not know how many there are.

§ 3 Proposal

We propose to extend the structured bindings syntax to allow the user to introduce a pack as (at most) one of the identifiers:

```
std::tuple<X, Y, Z> f();

auto [x,y,z] = f();           // OK today
auto [...xs] = f();           // proposed: xs is a pack of length three
                               containing an X, Y, and a Z
auto [x, ...rest] = f();      // proposed: x is an X, rest is a pack of length
                               two (Y and Z)
auto [x,y,z, ...rest] = f();  // proposed: rest is an empty pack
auto [x, ...rest, z] = f();   // proposed: x is an X, rest is a pack of length
                               one
                               // consisting of the Y, z is a Z
auto [...a, ...b] = f();      // ill-formed: multiple packs
```

If we additionally add the structured binding customization machinery to `std::integer_sequence`, this could greatly simplify generic code:

Today	Proposed
Implementing <code>std::apply()</code>	
<pre>namespace detail { template <class F, class Tuple, std::size_t... I> constexpr decltype(auto) apply_impl(F &&f, Tuple &&t, std::index_sequence<I...>) { return std::invoke(std::forward<F>(f), std::get<I>(std::forward<Tuple>(t))...); } } template <class F, class Tuple> constexpr decltype(auto) apply(F &&f, Tuple &&t) { return detail::apply_impl(std::forward<F>(f), std::forward<Tuple>(t), std::make_index_sequence<std::tuple_size_v< std::decay_t<Tuple>>>{}); }</pre>	<pre>template <class F, class Tuple> constexpr decltype(auto) apply(F &&f, Tuple &&t) { auto&& [...elems] = t; return std::invoke(std::forward<F> (f), forward_like<Tuple, decltype(elems)> (elems)...); }</pre>
<code>dot_product()</code> , nested	
<pre>template <class P, class Q> auto dot_product(P p, Q q) { return std::apply([&](auto... p_elems){ return std::apply([&](auto... q_elems){ return (... + (p_elems * q_elems)); }, q) }, p); }</pre>	<pre>template <class P, class Q> auto dot_product(P p, Q q) { // no indirection! auto&& [...p_elems] = p; auto&& [...q_elems] = q; return (... + (p_elems * q_elems)); }</pre>
<code>dot_product()</code> , with <code>index_sequence</code>	

```

template <size_t... Is, class P, class Q>
auto dot_product(std::index_sequence<Is...>, P p, Q,
               q) {
    return (... + (std::get<Is>(p) * std::get<Is>
                    (q)));
}

template <class P, class Q>
auto dot_product(P p, Q q) {
    return dot_product(
        std::make_index_sequence<std::tuple_size_v<P>>
        {},
        p, q);
}

```

```

template <class P, class Q>
auto dot_product(P p, Q q) {
    // no helper function
    // necessary!
    auto [...Is] =
        std::make_index_sequence<
        std::tuple_size_v<P>>{};
    return (... + (std::get<Is>(p)
                  * std::get<Is>(q)));
}

```

Not only are these implementations more concise, but they are also more functional. I can just as easily use `apply()` with user-defined types as I can with `std::tuple`:

```

struct Point {
    int x, y, z;
};

Point getPoint();
double calc(int, int, int);

double result = std::apply(calc, getPoint()); // ill-formed today, ok with
                                              proposed implementation

```

§ 3.1 Other Languages

Python 2 had always allowed for a syntax similar to C++17 structured bindings, where you have to provide all the identifiers:

```

>>> a, b, c, d, e = range(5) # ok
>>> a, *b = range(3)
File "<stdin>", line 1
    a, *b = range(3)
      ^
SyntaxError: invalid syntax

```

But you could not do any more than that. Python 3 went one step further by way of PEP-3132 [\[PEP.3132\]](#). That proposal allowed for a single starred identifier to be used, which would bind to all the elements as necessary:

```

>>> a, *b, c = range(5)
>>> a
0
>>> c
4
>>> b
[1, 2, 3]

```

The Python 3 behavior is synonymous with what is being proposed here. Notably, from that PEP:

Possible changes discussed were:

- Only allow a starred expression as the last item in the exprlist. This would simplify the unpacking code a bit and allow for the starred expression to be assigned an iterator. This behavior was rejected because it would be too surprising.

R0 of this proposal only allowed a pack to be introduced as the last item, which was changed in R1.

§ 3.2 Implementation Burden

Unfortunately, this proposal has some implementation complexity. The issue is not so much this aspect:

```
template <typename Tuple>
auto sum_template(Tuple tuple) {
    auto [...elems] = tuple;
    return (... + elems);
}
```

This part is more or less straightforward - we have a dependent type and we introduce a pack from it, but we're already in a template context where dealing with packs is just a normal thing.

The problem is this aspect:

```
auto sum_non_template(SomeConcreteType tuple) {
    auto [...elems] = tuple;
    return (... + elems);
}
```

We have not yet in the history of C++ had this notion of packs outside of dependent contexts. This is completely novel, and imposes a burden on implementations to have to track packs outside of templates where they previously had not.

However, in our estimation, this functionality is going to come to C++ in one form or other fairly soon. Reflection, in the latest form of [\[P1240R2\]](#), has many examples of introducing packs in non-template contexts as well - through the notion of a *reflection range*. That paper introduces several reifiers that can manipulate a newly-introduced pack, such as:

```
std::meta::info t_args[] = { ^int, ^42 };
template<typename T, T> struct X {};
X<...[:t_args:]...> x; // Same as "X<int, 42> x;".
template<typename, typename> struct Y {};
Y<...[:t_args:]...> y; // Error: same as "Y<int, 42> y;".
```

As with the structured bindings example in this paper - we have a non-dependent object outside of a template that we're using to introduce a pack.

Furthermore, unlike some of the reflection examples, and some of the more generic pack facilities proposed in [\[P1858R2\]](#), this paper offers a nice benefit: all packs must still be declared before use. Even in the

`sum_non_template` example which, as the name suggests, is not a template in any way, the pack `elems` needs an initial declaration. So any machinery that implementations need to track packs doesn't need to be enabled everywhere - only when a pack declaration has been seen.

§ 3.3 Implementation Experience

Jason Rice has implemented this in a [clang](#). As far as we've been able to ascertain, it works great.

It is also [available on Compiler Explorer](#).

§ 3.4 Handling non-dependent packs in the wording

The strategy the wording takes to handle is to introduce the concept of an “implicit template region.” A structured binding pack declaration introduces an implicit template region (if it's not already a templated entity), so that all entities declared within that scope are templated. This is important for a bunch of examples that we'll shortly see. That implicit template region is instantiated at the end of the region.

Example from Richard Smith:

```
template<int> struct X { using type = int; };
template<typename T> void f() {
    struct Y { int a, b; };
    auto [...v] = Y();
    X<sizeof...(v)>::type x; // need typename or no?
}
```

Is `X<sizeof...(v)>` a dependent type or is the pack `v` expanded eagerly because we already know its size? In this case we say a `sizeof...(e)` expression is *value-dependent* unless its referring to a structured binding pack whose initializer is not dependent. `Y()` isn't dependent, so `sizeof...(v)` isn't value dependent, so we don't need `typename` here.

That rule likewise handles this case:

```
struct Z { int a, b; };
template <typename T> void g() {
    auto [...v] = Z();
    X<sizeof...(v)>::type x; // typename unnecessary
}
```

§ 3.4.1 The Issaquah Example

Here is an interesting example, courtesy of Christof Meerwald:

```
struct C
{
    int i;
    long l;
};

template<typename T>
```

```

struct D {
    template<int J>
    static int f(int);
};

template<>
struct D<long> {
    static int f;
};

auto [ ... v ] = C{ 1, 0 };
auto x = ( ... + (D<decltype(v)>::f<1>(2)) );

```

This example demonstrates the need for having `typename` and/or `template` to disambiguate, even if we're not in a template context. `v` still needs to behave like a regular function parameter pack in this context - it still needs to be dependent.

§ 3.4.2 The Varna Example

Here is an interesting example, also courtesy of Christof Meerwald:

```

struct C { int j; long l; };

int g() {
    auto [ ... i ] = C{ 1, 2L };
    return ( [c = i] () {
        if constexpr (sizeof(c) > sizeof(long)) {
            static_assert(false); // error?
        }

        struct C {
            // error, not templated?
            int f() requires (sizeof(c) == sizeof(int)) { return 1; }

            // error, not templated?
            int f() requires (sizeof(c) != sizeof(int)) { return 2; }
        };
        return C{}.f();
    } () + ... + 0 );
}

// error?
int v = g();

```

What happens here? Core's intent is that this example is valid - the first `static_assert` declaration does not fire, this declares two different types `C`, both of which are valid, and `v` is initialized with the value `3`.

Here's an addendum from Jason Merrill, Jens Maurer, and John Spicer, slightly altered and somewhat gratuitously formatted hoping that it's possible to understand:

```

struct C { int j; long ; };

int g() {
    auto [ ... i ] = C{ 1, 2L };

```

```

if constexpr (sizeof...(i) == 0) {
    static_assert(false); // #1
}

return (
    // E1
    []{
        if constexpr (sizeof(int) == 0) {
            static_assert(false); // #2
        }
        return 1;
    })
    *
    // E2
    [c=i]{
        if constexpr (sizeof(c) > sizeof(long)) {
            static_assert(false); // #3
        }

        if constexpr (sizeof(int) == 0) {
            static_assert(false); // #4
        }

        return 2;
    })
    + ... + 0)
}

```

The expression E1 (that first immediately-invoked lambda) is completely non-dependent, no template packs, no structured binding packs, no packs of any kind. The expression E2 is an abridged version of the previous example's lambda - it does depend on the structured binding pack `i`.

The intent is that the `static_assert` declarations in #1 and #3 do not fire (since `i` is not an empty pack and neither `int` nor `long` has larger size than `long`), and this matches user expectation. The `static_assert` in #2, if written in a regular function, would immediately fire - since it's not guarded by any kind of template. So the questions are:

1. Does #2 still fire in this context, or does it become attached to the pack somehow?
2. Does #4 still fire in this context? It's still exactly as non-dependent as it was before, but now it's in a context that's at least somewhat dependent - since we're in a pack expansion over `...i`

This brings up the question of what the boundary of dependence is.

§ 3.4.3 More Templates

The approach suggested by John Spicer is as follows. Consider this example (let `si` just be some statements):

```

void foo() {
    auto [...xs] = C();
    s0;
    s1;
}

```

```
s2;  
}
```

Those statements get treated roughly as if they appeared in this context:

```
void foo() {  
    auto [...xs] = C();  
    [&](auto... xs){  
        s0;  
        s1;  
        s2;  
    }(xs...);  
}
```

Of course, not exactly like that (we're not literally introducing a generic lambda, there's no added function scope, all of these statements are directly in `foo` so that `returns` work, etc.). But this is the model: a structured binding pack, if not already a templated entity, introduces an *implicit template region*.

Importantly, it helps answer all of the questions in the previous examples: do the `static_asserts` fire in the [Varna example addendum](#)? No, none of them fire.

§ 3.5 Namespace-scope packs

In addition to non-dependent packs, this paper also seems like it would offer the ability to declare a pack at *namespace scope*:

```
struct Point { int x, y; };  
auto [... parts] = Point{1, 2};
```

Structured bindings in namespace scope are a little odd to begin with, since they currently cannot be declared `inline`. A structured binding pack at namespace scope adds that much more complexity.

Now, EWG originally [voted to remove this restriction](#) in a telecon - on the basis that it seemed like an artificial restriction and that the rules for how to use this feature should be uniform across all uses of structured bindings. However, during Core wording review of this paper in the Tokyo meeting, a lot of issues with this approach surfaced.

Introducing a structured binding pack into a function makes the region of that function into a template. This has consequences, but those consequences are localized to *just* the part of *that* function that follows the structured binding declaration.

Introducing a structured binding pack at namespace scope makes *your entire program* into a template. With enormous downstream consequences. Because unlike a function where we have a clear end to the region (the end of the block scope), even closing the namespace isn't sufficient because the namespace could still be reopened again later! Code that you have in one header could *change lookup rules* if it is included after another header that happened to add a namespace-scope pack. Even if your header never referenced that pack. If you had any non-dependent `if constexpr` conditions, they suddenly become dependent, and those bodies suddenly can become discarded.

Perhaps there is a way to divine some way to word this feature such that this doesn't happen, but that seems to be a very large amount of work necessary that far exceeds the potential utility thereof. It is not clear if anybody wants

this, and it certainly does not seem worth delaying this paper further to attempt to add support for it.

Thus, this paper proposes that structured binding packs be limited to block scope. This decision was confirmed with EWG at the [Tokyo meeting](#).

§ 4 Wording

Add a drive-by fix to 7.5.6 [\[expr.prim.fold\]](#) after paragraph 3:

π A fold expression is a pack expansion.

Add a new grammar option for *simple-declaration* to 9.1 [\[dcl.pre\]](#) (note that this accounts for [\[P0609R3\]](#) by renaming the grammar productions prefixed with *attributed* to *sb*):

```
- attributed-identifier:
-   identifier attribute-specifier-seqopt
+ sb-identifier:
+   ...opt identifier attribute-specifier-seqopt
+
- attributed-identifier-list:
-   attributed-identifier
-   attributed-identifier-list, attributed-identifier
+ sb-identifier-list:
+   sb-identifier
+   sb-identifier-list, sb-identifier

structured-binding-declaration:
-   attribute-specifier-seqopt decl-specifier-seq ref-qualifieropt [
-       attributed-identifier-list ]
+   attribute-specifier-seqopt decl-specifier-seq ref-qualifieropt [ sb-
+       identifier-list ]
```

Change 9.1 [\[dcl.pre\]](#)/6:

6 A *simple-declaration* with a *structured-binding-declaration* is called a structured binding declaration ([\[dcl.struct.bind\]](#)). Each *decl-specifier* in the *decl-specifier-seq* shall be `static`, `thread_local`, `auto` ([\[dcl.spec.auto\]](#)), or a *cv-qualifier*. The declaration shall contain at most one *sb-identifier* whose *identifier* is preceded by an ellipsis. If the declaration contains any such *sb-identifier*, it shall inhabit a block scope.

Extend 9.3.4.6 [\[dcl.fct\]](#)/5:

5 The type of a function is determined using the following rules. The type of each parameter (including function parameter packs), is determined from its own *parameter-declaration* ([\[dcl.decl\]](#)). After determining the type of each parameter, any parameter of type “array of T” or of function type T is adjusted to be “pointer to T”. After producing the list of parameter types, any top-level *cv-qualifiers* modifying a parameter type are deleted when forming the function type. The

resulting list of transformed parameter types and the presence or absence of the ellipsis or a function parameter pack is the function's parameter-type-list.

[Note 3: This transformation does not affect the types of the parameters. For example, `int(*)` (`const int p`, `decltype(p)*`) and `int(*)` (`int`, `const int*`) are identical types. — end note]

[Example 1:

```
void f(char*); // #1
void f(char[]) {} // defines #1
void f(const char*) {} // OK, another overload
void f(char *const) {} // error: redefines #1

void g(char(*)[2]); // #2
void g(char[3][2]) {} // defines #2
void g(char[3][3]) {} // OK, another overload

void h(int x(const int)); // #3
void h(int (*)(int)) {} // defines #3

+ void k(int, int); // #4
+ void m() {
+   struct C { int i, j; };
+   auto [... elems] = C{};
+   void k(decltype(elems)...); // redeclares #4
+ }
```

— end example]

Change 9.6 [\[`decl.struct.bind`\]](#) paragraph 1:

- 1 A structured binding declaration introduces the *identifiers* $v_0, v_1, v_2, \dots, v_{N-1}$ of the ~~*attribute-identifier-list-list*~~ *sb-identifier-list* as names ([basic.scope.declarative]) ~~of structured bindings~~. The optional *attribute-specifier-seq* of an ~~*attributed-identifier*~~ *sb-identifier* appertains to the structured binding so introduced. *An sb-identifier that contains an ellipsis introduces a structured binding pack* ([temp.variadic]). *A structured binding is either an sb-identifier that does not contain an ellipsis or an element of a structured binding pack*. Let *cv* denote the *cv-qualifiers* in the *decl-specifier-seq*.

Introduce new paragraphs after 9.6 [\[`decl.struct.bind`\]](#) paragraph 1, introducing the terms “structured binding size” and SB_i :

- 1+1 The *structured binding size* of *E*, as defined below, is the number of structured bindings that need to be introduced by the structured binding declaration. If there is no structured binding pack, then the number of elements in the *sb-identifier-list* shall be equal to the structured binding size of *E*. Otherwise, the number of elements of the structured binding pack is the structured binding size of *E* less the number of non-pack elements in the *sb-identifier-list*; the number of non-pack elements shall be no more than the structured binding size of *E*.
- 1+2 Let SB_i denote the i^{th} structured binding in the structured binding declaration after expanding the structured binding pack, if any. [*Note*: If there is no structured binding pack, then SB_i denotes v_i . -

end note]

[*Example 2:*

```
struct C { int x, y, z; };

auto [a, b, c] = C(); // OK, SB0 is a, SB1 is b, and SB2 is c
auto [d, ...e] = C(); // OK, SB0 is d, the pack e (v1) contains two structured
    bindings: SB1 and SB2
auto [...f, g] = C(); // OK, the pack f (v0) contains two structured bindings:
    SB0 and SB1, and SB2 is g
auto [h, i, j, ...k] = C(); // OK, the pack k is empty
auto [l, m, n, o, ...p] = C(); // error: structured binding size is too small
```

— *end example*]

Change 9.6 [\[dcl.struct.bind\]](#) paragraph 3 to define a structured binding size and extend the example:

- 3 If E is an array type with element type T, ~~the number of elements in the attributed-identifier-list shall be~~ **the structured binding size of E is** equal to the number of elements of E. Each ~~v_i~~ **SB_i** is the name of an lvalue that refers to the element *i* of the array and whose type is T; the referenced type is T. [*Note:* The top-level cv-qualifiers of T are cv. — *end note*]

[*Example 3:*

```
auto f() -> int(&)[2];
auto [ x, y ] = f();           // x and y refer to elements in a copy of the
    array return value
auto& [ xr, yr ] = f();        // xr and yr refer to elements in the array
    referred to by f's return value

+ auto g() -> int(&)[4];
+ auto [a, ...b, c] = g();      // a names the first element of the array, b is
    a pack referring to the second and
+                               // third elements, and c names the fourth
    element
+ auto& [...e] = g();           // e is a pack referring to the four elements
    of the array
```

— *end example*]

Change 9.6 [\[dcl.struct.bind\]](#) paragraph 4 to define a structured binding size:

- 4 Otherwise, if the *qualified-id* `std::tuple_size<E>` names a complete type, the expression `std::tuple_size<E>::value` shall be a well-formed integral constant expression and the ~~number of elements in the attributed-identifier-list shall be~~ **structured binding size of E is** equal to the value of that expression. [...] Each ~~v_i~~ **SB_i** is the name of an lvalue of type T_{*i*} that refers to the object bound to r_{*i*}; the referenced type is T_{*i*}.

Change 9.6 [\[dcl.struct.bind\]](#) paragraph 5 to define a structured binding size:

- 5 Otherwise, all of E's non-static data members shall be direct members of E or of the same base class of E, well-formed when named as `e.name` in the context of the structured binding, E shall not have an anonymous union member, and the ~~number of elements in the attributed-identifier-list shall be~~ structured binding size of E is equal to the number of non-static data members of E. Designating the non-static data members of E as m_0 , m_1 , m_2 , . . . (in declaration order), each v_i **SB_i** is the name of an lvalue that refers to the member m_i of E and whose type is `cv Ti`, where T_i is the declared type of that member; the referenced type is `cv Ti`. The lvalue is a bit-field if that member is a bit-field.

Change 13.1 [\[temp.pre\]/8](#) to extend the notion of what is a templated entity, first introducing the term *implicit template region*:

- 7a An *implicit template region* is a subregion of a block scope. An implicit template region is a template definition. At the end of an implicit template region, it is immediately instantiated ([temp.pre]). A declaration of a structured binding pack ([dcl.struct.bind]) that is not a templated entity (see below) introduces an implicit template region that includes the declaration of the structured binding pack and ends at the end of the scope that the structured binding declaration inhabits.
- 8 An entity is *templated* if it is
- (8.1) — a template,
 - (8.2) — an entity defined ([basic.def]) or created ([class.temporary]) in a templated entity,
 - (8.3) — a member of a templated entity,
 - (8.4) — an enumerator for an enumeration that is a templated entity, ~~or~~
 - (8.5) — the closure type of a lambda-expression ([expr.prim.lambda.closure]) appearing in the declaration of a templated entity; ~~,~~ or
 - (8.6) — an entity defined or created within an implicit template region.

[Note 1: A local class, a local or block variable, or a friend function defined in a templated entity is a templated entity. — end note]

[Example 4: The following example assumes that `char` and `int` have different size.

```
struct C { char j; int l; };

int g() {
    auto [ ... i ] = C{ 'x', 42 }; // #1
    return ( [c = i] () {
        // The local class L is a templated entity because
        // it is defined within the implicit template region
        // created at #1.
        struct L {
            int f() requires (sizeof(c) == 1) { return 1; }
            int f() requires (sizeof(c) != 1) { return 2; }
        };
        return L{}.f();
    } () + ... + 0 );
```

```

}

int v = g(); // OK, v == 3
— end example ]
[ Example 5:

struct C { };

void g(...); // #1

template <typename T>
void f() {
    C arr[1];
    auto [...e] = arr;
    g(e...); // calls #2
}

void g(C); // #2

int main() {
    f<int>();
}
— end example ]

```

Add a new clause to 13.7.4 [\[temp.variadic\]](#), after paragraph 3:

- 3+ A *structured binding pack* is an *sb-identifier* that introduces zero or more structured bindings ([`dcl.struct.bind`]).

[Example 6:

```

auto foo() -> int(&)[2];
auto [...a] = foo();           // a is a structured binding pack containing 2
    elements
auto [b, c, ...d] = foo();     // d is a structured binding pack containing 0
    elements

```

— end example]

In 13.7.4 [\[temp.variadic\]](#), change paragraph 4:

- 4 A *pack* is a template parameter pack, a function parameter pack, ~~or~~ an *init-capture* pack, **or a structured binding pack**. The number of elements of a template parameter pack or a function parameter pack is the number of arguments provided for the parameter pack. The number of elements of an *init-capture* pack is the number of elements in the pack expansion of its *initializer*.

In 13.7.4 [\[temp.variadic\]](#), paragraph 5 (describing pack expansions) remains unchanged.

In 13.7.4 [\[temp.variadic\]](#), add a bullet to paragraph 8:

- 8 Such an element, in the context of the instantiation, is interpreted as follows:

- (8.1) — if the pack is a template parameter pack, the element is a template parameter ([temp.param]) of the corresponding kind (type or non-type) designating the i^{th} corresponding type or value template argument;
- (8.2) — if the pack is a function parameter pack, the element is an *id-expression* designating the i^{th} function parameter that resulted from instantiation of the function parameter pack declaration; **otherwise**
- (8.3) — if the pack is an *init-capture* pack, the element is an *id-expression* designating the variable introduced by the i^{th} *init-capture* that resulted from instantiation of the *init-capture* pack; **otherwise**
- (8.4) — **if the pack is a structured binding pack, the element is an *id-expression* designating the i^{th} structured binding in the pack that resulted from the structured binding declaration.**

Add a bullet to 13.8.3.3 [\[temp.dep.expr\]/3](#):

- 3 An *id-expression* is type-dependent if it is a *template-id* that is not a concept-id and is dependent; or if its terminal name is
- (3.1) — associated by name lookup with one or more declarations declared with a dependent type,
 - (3.2) — associated by name lookup with a non-type template-parameter declared with a type that contains a placeholder type,
 - (3.3) — associated by name lookup with a variable declared with a type that contains a placeholder type ([dcl.spec.auto]) where the initializer is type-dependent,
 - (3.4) — associated by name lookup with one or more declarations of member functions of a class that is the current instantiation declared with a return type that contains a placeholder type,
 - (3.5) — associated by name lookup with a structured binding declaration ([dcl.struct.bind]) whose brace-or-equal-initializer is type-dependent,
 - (3.5b) — **associated by name lookup with a pack, unless that pack is a non-type template parameter pack whose types are non-dependent,**
 - (3.6) — associated by name lookup with an entity captured by copy ([expr.prim.lambda.capture]) in a lambda-expression that has an explicit object parameter whose type is dependent ([dcl.fct]),
 - (3.7) — the identifier `__func__` ([dcl.fct.def.general]), where any enclosing function is a template, a member of a class template, or a generic lambda,
 - (3.8) — a conversion-function-id that specifies a dependent type, or
 - (3.9) — dependent

Add a carve-out for in 13.8.3.4 [\[temp.dep.constexpr\]/4](#):

- 4 Expressions of the following form are value-dependent:

```
sizeof ... ( identifier )
fold-expression
```

unless the *identifier* is a structured binding pack whose initializer is not dependent.

Add a new paragraph at the end of 13.8.4.1 [\[temp.point\]](#):

- * For an implicit template region, the point of instantiation immediately follows the namespace scope declaration or definition that encloses the region.

§ 4.1 Feature-Test Macro

Bump `__cpp_structured_bindings` in 15.11 [\[cpp.predefined\]](#):

```
- __cpp_structured_bindings 201606L  
+ __cpp_structured_bindings 2024XXL
```

§ 5 Acknowledgements

Thanks to Michael Park and Tomasz Kamiński for their helpful feedback. Thanks to Richard Smith for help with the wording. Thanks especially to Jason Rice for the implementation.

Thanks to John Spicer, Christof Meerwald, Jens Maurer, and everyone else in Core for the wording help, mind-melting examples, and getting this paper in shape.

§ 6 References

- [N3915] Peter Sommerlad. 2014-02-14. `apply()` call a function with arguments from a tuple (V3).
<https://wg21.link/n3915>
- [P0144R2] Herb Sutter. 2016-03-16. Structured Bindings.
<https://wg21.link/p0144r2>
- [P0609R3] Aaron Ballman. 2024-03-21. Attributes for Structured Bindings.
<https://wg21.link/p0609r3>
- [P1061R0] Barry Revzin, Jonathan Wakely. 2018-05-01. Structured Bindings can introduce a Pack.
<https://wg21.link/p1061r0>
- [P1061R0.Minutes] EWGI. 2019. Kona 2019 EWGI: P1061R0.
<http://wiki.edg.com/bin/view/Wg21kona2019/P1061>
- [P1061R1] Barry Revzin, Jonathan Wakely. 2019-10-07. Structured Bindings can introduce a Pack.
<https://wg21.link/p1061r1>
- [P1061R1.Minutes] EWG. 2019. Belfast 2020 EWG: P1061R1.
<https://wiki.edg.com/bin/view/Wg21belfast/P1061-EWG>
- [P1240R2] Daveed Vandevoorde, Wyatt Childers, Andrew Sutton, Faisal Vali. 2022-01-14. Scalable Reflection.
<https://wg21.link/p1240r2>
- [P1858R2] Barry Revzin. 2020-03-01. Generalized pack declaration and usage.
<https://wg21.link/p1858r2>
- [PEP.3132] Georg Brandl. 2007. PEP 3132 – Extended Iterable Unpacking.
<https://www.python.org/dev/peps/pep-3132/>